

Fluid Dynamics - Practicals

Sylvain MARTIN

25 juillet 2025

1 Implementation of a 2D SIMPLE solver

1.1 Presentation of the system

The aim of this practical is to implement a homemade SIMPLE solver in 2D in matlab to solve a Poiseuille flow, as shown in figure 1.1. The latter is a 2D laminar flow with the inlet flow at the left wall and an outflow at the right wall. The main advantage of this test case is that the analytical solution is known.

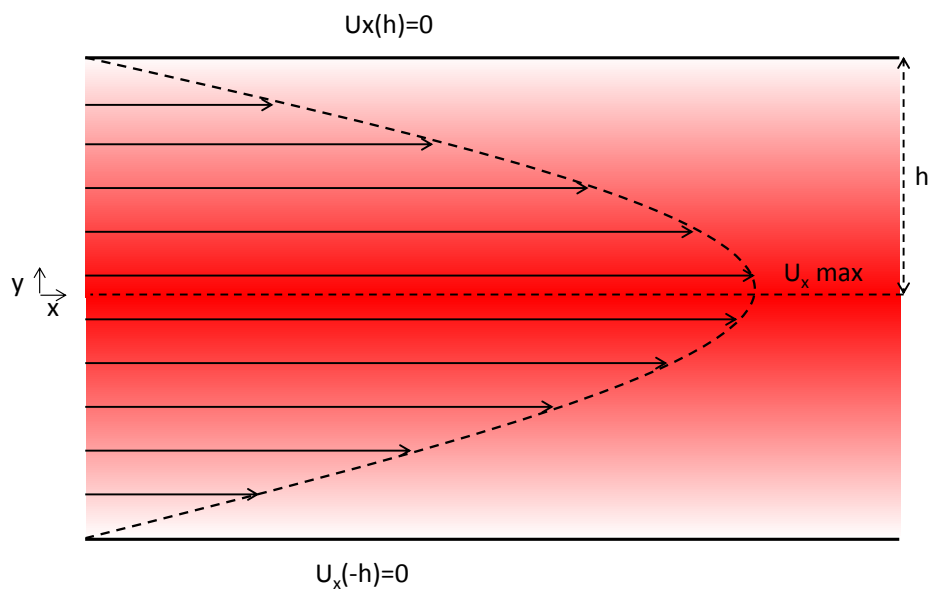


FIGURE 1.1 – Poiseuille flow

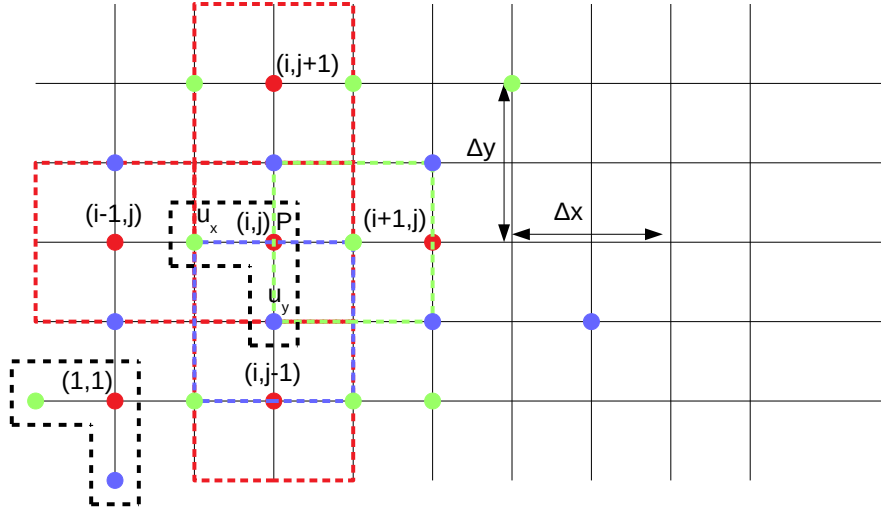


FIGURE 1.2 – Example of a staggered grid for a structured Cartesian mesh. Pressure is evaluated on red nodes, u_x is evaluated on green nodes and u_y on blue nodes.

1.2 Staggered grid

A rectangular grid is defined by its numbers of nodes in the x and y direction, respectively i_{max} and j_{max} . One first point that should be managed is the use of a staggered grid as shown in figure 1.2. If one wants to impose a clean boundary condition, it is necessary to add an extra line of green nodes (u_x) on the right and an extra line of blue nodes (u_y) on the top.

In python, the index in a table of size N goes from 0 to $N - 1$. Thus, u_x **grid should be defined from** $\{i = 0; i_{max}\}$ **and** $\{j = 0; j_{max} - 1\}$ **and** u_y **from** $\{i = 0; i_{max} - 1\}$ **and** $\{j = 0; j_{max}\}$.

Then, to get details in the full algorithm, one can have a look back to the class notes. The main steps are reminded in algo 1.

1.2.1 Boundary Conditions

For u_x

The boundary conditions are :

- North and South : No slip
- West : imposed inlet velocity
- East : outflow

Algorithm 1 SIMPLE algorithm for steady-state flow

```
Initial guess for  $P, u_x, u_y$ 
while Convergence = false do
    Calculate link coefficients and estimate of  $u_x^{k*}$ 
    Calculate link coefficients and estimate  $u_y^{k*}$ 
    Calculate link coefficients and estimate  $P^k$ 
    New estimate of  $u_x^k, u_y^k$ 
    Test convergence
     $k=k+1$ 
end while
```

Thus, u_x velocity can be set such as $u_x = \text{left_velocity}$ at the western wall. At the eastern wall, a good compromise is to impose no constraint, i.e. no velocity gradient. Hence, $u_x(\text{imax}) = u_x(\text{imax} - 1)$. Conversely, at the northern and southern walls, boundary conditions for u_x should be derived from a Finite volume balance around the boundary nodes (see lecture notes).

For u_y

- The boundary conditions for u_y are as follows :
- North, South and West : null
 - East outflow

u_y can be imposed at North and South wall such as $u_y = 0$ to match with the no slip boundary. At the East wall, the zero constraints leads to $u_y(\text{jmax}) = u_y(\text{jmax} - 1)$. At the western wall, we also need to implement $u_y = 0$. Then the value of the first range of u_y should be derived from a Finite Volume balance at the wall.

For the pressure P

Finally, the Pressure at the wall should be derived from Finite Volume balance at North, South and West wall. At the outflow (Eastern wall), one can set the reference pressure, such as $P = 0$.

1.3 Presentation of the code

The structure of the code matlab code is given. It is composed of 6 files :

- *Simple_Main* which contains the preprocessing, post processing and the main loop.
- *solve_x_momentum* which deals with the calculation of u_x^*
- *solve_y_momentum* which deals with the calculation of u_y^*
- *solve_P* which calculates P' (labeled *Delta_P* in the code)
- *corect_uX* which deals with the calculation of u_x^k
- *corect_uY* which deals with the calculation of u_y^k

1.4 Inner solvers

The calculation of u_x^* , u_y^* and P' requires to solve a linear system. It could be tempting to write the system under the form of a matrix form and use standard python functions to solved. However, it should be reminded that, u_x^* and u_y^* for example are not the exact solution of the NS equation but new approximation a bit closer from the solution of the linearized momentum equation. Hence, there is no need for an accurate solver. As a reminder, the discrete linearized x momentum equation is :

$$a_{x,i,j}u_{x,i,j}^* + a_{x,i-1,j}u_{x,i-1,j}^* + a_{x,i+1,j}u_{x,i+1,j}^* + a_{x,i,j-1}u_{x,i,j-1}^* + a_{x,i,j+1}u_{x,i,j+1}^* = A_{x,i,j}P_{i,j} + A_{i-1,j}P_{-1i,j}$$

which is indeed a linear system. In this pratical, it was chosen to use the **Jacobi** algorithm, which is :

- very simple,
- easy to vectorize if necessary.

It is an iterative algorithm. Thus, the values must be inisialized, basically $u_x^{*0} = u_x^k$. Then, the estimation at rank m can easily be estimated from the known values at rank $m - 1$. For the x momentum equation, this leads to :

$$u_{x,i,j}^{*m} = \frac{1}{a_{x,i,j}} (-a_{x,i-1,j}u_{x,i-1,j}^{*m-1} - a_{x,i+1,j}u_{x,i+1,j}^{*m-1} - a_{x,i,j-1}u_{x,i,j-1}^{*m-1} - a_{x,i,j+1}u_{x,i,j+1}^{*m-1} + A_{x,i,j}P_{i,j} + A_{i-1,j}P_{-1i,j})$$

where m is the Jacobi solver counter (inner iterations).

he node position if there is not enough iterations.

This approach can seem very approximative. However, it should be noticed that there is no interest to solve it until machine accuracy since u^* is only a new estimation which does not respect the continuity equation and that will be corrected afterwards. Hence, a quick improvement of previous solution is enough and it is exactly what is provided by iterative methods.

The Jacobi solver is only and option, you are free to implement any other linear solver.

1.5 Road map

The coming steps are given to guide you through the SIMPLE solver programming.

1. Fills in the link coefficients for x momentum, in the bulk and boundaries.
2. Fills in the link coefficients for y momentum, in the bulk and boundaries.
3. Program the linear solver for x and y momentum and run the code. Do the results look consistent ?
4. Fills in the link coefficients for Pressure
5. Program the linear solver for Pressure correction

6. Program the x, y velocity corrections.
7. Validate your code for standard (already programmed case)
8. Play with the solver parameter and flow conditions (Reynolds number)
9. If there is time left, have a look at the corner node and try to implement a proper boundary condition.

The report should detail the implementation and validation steps, explain the main difficulties and how you overcome.

1.6 Post processing

The figures are automatically shown and registered in the Results folder. They allow to check the convergence of velocity and pressure fields toward the analytical solution.

Enjoy, and be curious !